

# Staff User Stories

---

## *Team Project Phase 3 (TP3)*

**Purpose.** This document expands the staff-facing TP3 epics into a cohesive set of user stories. The stories are written to fit the current codebase direction: invitation-based account setup, role-based staff tools, admin user management, discussion review, abuse-report review, and participation-analysis support.

### Scope and assumptions

- Primary actor for these stories is a staff user operating as an Admin or another instructional-team role with staff permissions.
- Highest priority is given to stories that are already supported or clearly suggested by the current TP3 code branch, so the final execution screencast can demonstrate realistic behavior.
- One derived analytics story is included to align with the HW3 participation-analysis prototype and the TP3 requirement that staff be able to assess discussion participation more fairly.

### User stories

#### STF-01 - Invite a new participant into the system

**User Story.** As an instructional-team member acting as an Admin, I want to send an invitation code tied to an email address and role, so that new participants can join the system through a controlled onboarding process.

**Source / Kind.** Received/derived from staff epics and current TP3 admin workflow.

**Rationale.** The current admin home page includes an invitation area, role selection, outstanding invitation count, and code generation tied to email + role.

##### Acceptance notes

- The admin can enter an email address and choose a role before sending the invitation.
- The system validates the email format and blocks reuse of an already-invited address.
- The system generates a code, stores the outstanding invitation, and updates the visible invitation count.

#### STF-02 - See current administrative status at a glance

**User Story.** As an instructional-team member acting as an Admin, I want to see basic system status such as outstanding invitations and number of users, so that I can quickly understand the current operating state before taking action.

**Source / Kind.** Derived story supporting staff operational awareness.

**Rationale.** The current admin home page already presents counts for invitations and users, making this a natural staff requirement rather than only a UI convenience.

##### Acceptance notes

- The admin home page shows the current number of users.
- The admin home page shows the current number of outstanding invitations.
- These values refresh after invitation-related actions that change system state.

### STF-03 - Review users with role and active-status filters

**User Story.** As an instructional-team member acting as an Admin, I want to review a list of users and filter it by role and active status, so that I can quickly locate the accounts that need attention.

**Source / Kind.** Received/derived from staff-management epics.

**Rationale.** The admin user dialog exposes a table of user IDs, display names, roles, and active state, along with optional role and active-only filters.

#### Acceptance notes

- The staff user can open a user-management view from the admin area.
- The system displays each user with a visible role and active/inactive status.
- The staff user can narrow the displayed list using at least a role filter and an active-only filter.

### STF-04 - Activate or deactivate a user account

**User Story.** As an instructional-team member acting as an Admin, I want to activate or deactivate a user account, so that I can temporarily suspend access or restore it when needed.

**Source / Kind.** Received/derived from staff control and account-governance epics.

**Rationale.** The admin user dialog provides explicit Activate and Deactivate actions, and the database/service layer tracks accountActive state.

#### Acceptance notes

- The staff user can select an account and request activation or deactivation.
- The system clearly indicates whether the operation succeeded or failed.
- The updated account state is visible after the action is completed.

### STF-05 - Change a user's assigned role safely

**User Story.** As an instructional-team member acting as an Admin, I want to change a user's assigned role, so that the system reflects the responsibilities the user should currently have.

**Source / Kind.** Received story tied to staff administration.

**Rationale.** The codebase now contains both an Add/Remove Roles page and an admin-service dialog that changes roles while protecting the last active Admin.

#### Acceptance notes

- The staff user can choose a target user and select a replacement or additional role from the permitted role set.
- Unauthorized actors must not be able to grant themselves or others a new role.
- The system must prevent removal of the last active Admin account.

### STF-06 - Reset a user password with a forced follow-up change

**User Story.** As an instructional-team member acting as an Admin, I want to reset a user's password and mark the account for a follow-up password change, so that I can recover access safely without permanently knowing the user's chosen password.

**Source / Kind.** Received/derived from staff account-recovery epics.

**Rationale.** The admin workflow includes reset-password support through the service layer, temporary password generation, and a must-reset flag in the user database.

**Acceptance notes**

- The staff user can trigger a password reset for a selected account.
- The system generates a temporary password rather than requiring the staff user to invent one manually.
- The account is marked so the affected user must change the password after the reset event.

## STF-07 - Inspect discussion activity across all posts

**User Story.** As an instructional-team member, I want to browse all discussion posts, search by text, and filter by solved status, so that I can quickly review the state of ongoing discussions.

**Source / Kind.** Derived staff story built on the shared discussion-board view.

**Rationale.** The discussion board currently supports a searchable, filterable post list with solved/unsolved status and detailed thread viewing.

**Acceptance notes**

- The staff user can open a complete list of discussion posts.
- The system supports text search over the post list.
- The system supports filtering between all, solved, and unsolved discussion threads.

## STF-08 - Inspect a full discussion thread in context

**User Story.** As an instructional-team member, I want to open a discussion thread and inspect the post, its replies, the solved state, and the involved users, so that I can evaluate the quality and outcome of the conversation in context.

**Source / Kind.** Received/derived from staff discussion-review epics.

**Rationale.** The thread view shows the original post body, formatted metadata, replies, solved/re-opened state, and available moderation/review actions.

**Acceptance notes**

- The staff user can open a single thread from the post list.
- The system shows the original post and its replies together in one review view.
- The current resolved/open state of the thread is visible while reviewing the conversation.

## STF-09 - Review and inspect reported discussion content

**User Story.** As an instructional-team member, I want to see the set of active abuse reports and inspect the offending post or reply in context, so that I can decide whether staff intervention is needed.

**Source / Kind.** Received/derived from moderation and quality-control epics.

**Rationale.** The database stores reports, the controller loads them, and the discussion-board UI provides a dedicated reports view with report count, offending user, summary, and an inspect action.

**Acceptance notes**

- The staff user can open a dedicated reports view from the admin area.
- Each active report identifies the reported user-generated content and how many reports it has received.
- The staff user can inspect the underlying content in context before deciding how to proceed.

## STF-10 - Assess whether a student answered enough different peers

**User Story.** As an instructional-team member, I want the system to summarize whether each student replied to at least three different peers, so that I can assess participation more fairly and with less manual counting.

**Source / Kind.** Derived story from the HW3 prototype and TP3 staff-assessment goal.

**Rationale.** The HW3 analytics tests define a participation-analysis direction based on counting distinct peers, ignoring self-answers, preventing duplicate inflation, and identifying students above or below a configurable threshold.

### Acceptance notes

- The system calculates unique-peer coverage rather than simply counting raw reply totals.
- Self-replies and duplicate replies to the same peer do not incorrectly inflate the participation count.
- The staff user can identify which students currently meet the threshold, which do not, and how many peers are still missing.

## STF-11 - Enforce staff-only actions and protected operations

**User Story.** As a stakeholder concerned with fair and safe operations, I want staff-only actions such as role changes, administrative password resets, and report review to be protected by authorization checks, so that students or other unauthorized users cannot interfere with assessment workflows.

**Source / Kind.** Derived security story required to support all other staff stories.

**Rationale.** The TP3 branch includes explicit authorization tests for admin services and database update operations, showing that protected staff workflows are part of the intended design rather than an afterthought.

### Acceptance notes

- Unauthorized users cannot grant roles, reset another user's password, or perform equivalent staff-only actions.
- Authorized staff users can complete those actions through the intended admin workflows.
- The protection of staff-only actions is validated by automated tests as well as normal execution behavior.